

BİLGİSAYAR MÜHENDİSLİĞİ

Programlama Dillerinin Kavramları

Bölüm 5

İsimler, Bağlamalar ve Kapsam

5.1 Giriş | 5.2 İsimler | 5.3 Değişkenler | 5.4 Bağlama Kavramı

Bu Belgede Kapsanan Konular

- ✦ 5.1 Giriş — Programlama dillerinde değişken kavramının temeli
- ✦ 5.2 İsimler — Tasarım soruları, isim biçimleri, büyük/küçük harf hassasiyeti, özel kelimeler
- ✦ 5.3 Değişkenler — Ad, adres, tür, değer, alias (takma ad) kavramları
- ✦ 5.4 Bağlama Kavramı — Bağlama zamanları, tür bağlama, depolama bağlama ve ömür kategorileri

Kaynak: Robert W. Sebesta, *Concepts of Programming Languages*, 12. Baskı, Bölüm 5

5.1 Giriş

Introduction

Zorunlu (imperative) programlama dilleri, temel olarak von Neumann bilgisayar mimarisinin soyutlamalarıdır. Bu mimarinin iki ana bileşeni vardır: hem komutları hem de verileri saklayan bellek ve belleğin içeriğini değiştirme işlemlerini sağlayan işlemci. Programlama dillerindeki makinenin bellek hücrelerine yönelik soyutlamalar değişkenlerdir (variables).

Bir değişken; tür (type), kapsam (scope) ve ömür (lifetime) gibi özellikler (attributes) koleksiyonuyla karakterize edilebilir. Bu özelliklerden en önemlisi türdür. Fonksiyonel programlama dillerinde ifadeler isimlendirilebilir; ancak bunlar değiştirilemez oldukları için zorunlu dillerdeki değişkenlerden farklıdır — bu tür isimlendirmeler daha çok adlandırılmış sabitlere (named constants) benzer.

□ Bölüm 5'in Özeti

Bu bölüm; isimlerin doğasını, değişken özelliklerini (tür, adres, değer), alias (takma ad) kavramını, bağlama (binding) ve bağlama zamanlarını, statik ve dinamik kapsam kurallarını ve adlandırılmış sabitleri kapsamlı biçimde ele almaktadır.

5.2 İsimler

Names

Değişkenler tartışılmadan önce isimlerin tasarımı incelenmelidir. İsimler; alt programlar, biçimsel parametreler ve diğer program yapıları ile de ilişkilidir. Tanımlayıcı (identifier) terimi çoğunlukla isimle birbirinin yerine kullanılır.

5.2.1 Tasarım Soruları

□ İsimler İçin Temel Tasarım Soruları

1. İsimler büyük/küçük harf duyarlı mı? (case sensitive)
2. Dilin özel kelimeleri ayrılmış kelimeler mi (reserved words) yoksa anahtar kelimeler mi (keywords)?

5.2.2 İsim Biçimleri

İsim, bir programda bir varlığı tanımlamak için kullanılan karakter dizisidir. Çoğu programlama dilinde isimler aynı forma sahiptir: bir harf ile başlar ve ardından harf, rakam ve alt çizgi (_) karakterlerinden oluşan bir dizi gelir.

Dillere Göre İsim Uzunluk Sınırları

Dil	İç İsim Sınırı	Dış İsim Sınırı	Not
C99	Sınırsız (ama 63 anlamlı)	31 karakter	Linker kısıtlaması
C++	Belirtilmemiş	Belirtilmemiş	Derleyiciye bağlı
Java	Sınırsız	Sınırsız	Tümü anlamlı
C#	Sınırsız	Sınırsız	Tümü anlamlı
Fortran I	6 karakter	6 karakter	Tarihsel kısıt

Camel Notation (Deve Notasyonu): C-tabanlı dillerde alt çizgi yerine çok sözcüklü isimlerde ilk sözcük hariç diğerlerinin baş harfleri büyük yazılır. Örneğin: `myStack`, `totalCount`, `pageWidth`.

Bazı dillerde isimlerin başındaki özel karakterler tür bilgisi taşır:

Dil	Özel Karakter	Anlamı
PHP	\$	Tüm değişkenler \$ ile başlar
Perl	\$ @ %	\$ → skalar, @ → dizi, % → hash yapısı
Ruby	@ @@	@ → örnek (instance) değişkeni, @@ → sınıf (class) değişkeni

5.2.3 Büyük/Küçük Harf Duyarlılığı (Case Sensitivity)

C-tabanlı birçok dilde büyük ve küçük harfler farklıdır; yani isimler büyük/küçük harf duyarlıdır (case sensitive). Örneğin C++'da `rose`, `R0SE` ve `Rose` birbirinden tamamen farklı üç isimdir.

Özellik	Büyük/Küçük Harf Duyarlı	Büyük/Küçük Harf Duyarsız
Okunabilirlik	Benzer görünen isimler farklı anlama gelebilir → DÜŞÜK	Daha tutarlı → YÜKSEK
Yazılabilirlik	Spesifik büyük/küçük harf kullanımı gerektirir → DÜŞÜK	Daha esnek → YÜKSEK
Örnek diller	C, C++, Java, C#, JavaScript	Fortran, Pascal
Problem	parseInt ≠ ParseInt ≠ parseint	Yok

⚠ Case Sensitivity Tasarım Sorunu

Java ve C#'da pek çok ön tanımlı isim büyük ve küçük harf karışımı içerir. Örneğin parseInt metodu. Bu durum yazılabilirlik (writability) sorununa yol açar: programcının hangi harfin büyük hangisinin küçük olduğunu ezberlemesi gerekir. Bu, dil tasarımcısının bir tür hoşgörüsüzlüğüdür ve derleyici tarafından zorlanır.

5.2.4 Özel Kelimeler (Special Words)

Programlama dillerindeki özel kelimeler; programları daha okunabilir kılmak, gerçekleştirilecek işlemleri adlandırmak ve deyimlerin ile programların sözdizimsel bölümlerini ayırmak için kullanılır.

Kavram	Tanım	Örnek
Ayrılmış Kelime (Reserved Word)	Programcı tarafından yeniden tanımlanamaz; her zaman özel anlamını korur.	if, while, for, int (C'de)
Anahtar Kelime (Keyword)	Belirli bağlamlarda özel anlam taşır; programcı tarafından yeniden tanımlanabilir.	Fortran'daki INTEGER, REAL gibi kelimeler
Ön Tanımlı İsim (Predefined Name)	Diğer program birimlerinde (paket, kütüphane) tanımlı; açıkça import edilmeli.	Java paketleri, C kütüphane isimleri

❏ COBOL Örneği — Fazla Ayrılmış Kelime Sorunu

COBOL'da 300'den fazla ayrılmış kelime bulunmaktadır. Bu sayı o kadar fazladır ki programcıların sıklıkla kullanmak istediği LENGTH, BOTTOM, DESTINATION, COUNT gibi kelimeler bile ayrılmış kelimeler listesindedir. Bu, ayrılmış kelimelerin aşırı sayıda olmasının yarattığı tipik bir yazılabilirlik sorunudur.

5.3 Değişkenler

Variables

Bir program değişkeni, bir bilgisayar bellek hücresinin veya hücre koleksiyonunun soyutlamasıdır. Makine dilinden assembly diline geçişin en büyük faydası, sayısal mutlak bellek adreslerinin isimlerle değiştirilmesiydi — bu durum programları çok daha okunabilir ve bakımı kolay hale getirdi.

□ Değişkenin Altı Özelliği (Sextuple)

Bir değişken şu altı özellik (attribute) ile karakterize edilir:

(isim, adres, değer, tür, ömür, kapsam)

Bu altılı; değişkenlerin farklı yönlerini açıklamanın en net yoludur.

5.3.1 İsim (Name)

Değişken isimleri programlardaki en yaygın isimlerdir. İsimler, 5.2. bölümde program varlıklarının genel bağlamında ayrıntılı biçimde ele alınmıştır. Bazı değişkenlerin isimleri yoktur — bunlar dinamik depolamayla ilgili bölümlerde incelenecektir.

5.3.2 Adres (Address)

Değişkenin adresi, onunla ilişkilendirildiği makine belleği adresidir. Birçok dilde, aynı değişkenin program yürütme sırasında farklı zamanlarda farklı adreslerle ilişkilendirilebileceğine dikkat edilmelidir.

Örneğin, bir alt programın çalışma zamanı yığınınından (run-time stack) tahsis edilen yerel bir değişkeni, farklı çağrılarda farklı adreslere sahip olabilir. Bunlar aynı değişkenin farklı örnekleri olarak düşünülebilir.

Kavram	Açıklama
l-değeri (l-value)	Değişkenin adresidir. İsim bir atama ifadesinin sol tarafında görüldüğünde gereklidir.
r-değeri (r-value)	Değişkenin değeridir. İsim bir atama ifadesinin sağ tarafında görüldüğünde gereklidir.

5.3.3 Alias (Takma Ad) Kavramı

Aynı bellek konumuna erişmek için birden fazla değişken ismi kullanılabildiğinde bu değişkenler alias (takma ad) olarak adlandırılır.

⚠ Alias Neden Zararlıdır?

Aliasing, okunabilirliği engeller çünkü bir değişkenin değeri farklı bir değişkene yapılan atama ile değiştirilebilir. Örneğin total ve sum alias ise, total'a yapılan değişiklik sum'ı da etkiler. Bu durum program doğrulamasını (verification) da güçleştirir.

Alias Oluşturma Yolları:

Yol	Açıklama	Örnek
Union Türleri	C ve C++'da union türleri alias oluşturabilir	union { int i; float f; }
Pointer Değişkenler	İki pointer aynı konuma işaret ediyorsa alias oluşur	int *p1 = &x; int *p2 = &x;
Referans Değişkenler	C++ referansları alias oluşturur	int &ref; = x; // ref ve x alias
Alt Program Parametreleri	Bazı parametre geçirme yöntemleri alias yaratabilir	Bölüm 9'da incelenecek

5.3.4 Tür (Type)

Bir değişkenin türü, değişkenin depolayabileceği değerlerin aralığını ve bu türdeki değerler için tanımlanmış işlem kümesini belirler. Örneğin Java'daki int türü -2.147.483.648'den 2.147.483.647'ye kadar bir değer aralığı ve toplama, çıkarma, çarpma, bölme ve modül işlemlerini tanımlar.

5.3.5 Değer (Value)

Değişkenin değeri, onunla ilişkilendirilen bellek hücresinin veya hücrelerinin içeriğidir. Bilgisayar belleğini fiziksel hücreler yerine soyut hücreler (abstract cells) cinsinden düşünmek daha yararlıdır.

□ Soyut Bellek Hücresi

Fiziksel bellek hücreleri 8-bit'lik baytlardır. Ancak bir değişkenin değeri soyut olarak tek bir hücre kaplar. Örneğin bir float değeri gerçekte 4 bayt kaplar, ancak soyut olarak tek bir hücre olarak düşünülür. r-değerine erişmek için önce l-değerinin belirlenmesi gerekir.

5.4 Bağlama Kavramı

The Concept of Binding

Bir bağlama (binding), bir özellik (attribute) ile bir varlık (entity) arasındaki ilişkilendirmedir. Örneğin bir değişkenle türü veya değeri arasındaki, ya da bir işlemle sembolü arasındaki ilişki birer bağlamadır. Bağlamanın gerçekleştiği zamana bağlama zamanı (binding time) denir.

□ Bağlama Zamanları — Ne Zaman Bağlanır?

1. Dil Tasarım Zamanı (Language Design Time): * sembolü çarpma işlemine bağlanır
2. Dil Gerçekleştirme Zamanı (Language Implementation Time): C'deki int türünün değer aralığı belirlenir
3. Derleme Zamanı (Compile Time): Java'da bir değişken belirli bir veri türüne bağlanır
4. Yükleme Zamanı (Load Time): Global değişken bir bellek hücresine bağlanabilir
5. Bağlama Zamanı (Link Time): Kütüphane alt programına yapılan çağrı bağlanır
6. Çalışma Zamanı (Run Time): Java metodunda yerel değişken depolamaya bağlanır

Örnek: `count = count + 5;` — **C++ Atama Deyimi**

Öge	Bağlama	Bağlama Zamanı
count değişkeninin türü	int türüne bağlanır	Derleme zamanı
count'un olası değer kümesi	int aralığına bağlanır	Derleyici tasarım zamanı
+ operatör sembolünün anlamı	Tamsayı toplamasına bağlanır (operand türleri bilinince)	Derleme zamanı
5 literalinin iç gösterimi	İkili tamsayıya bağlanır	Derleyici tasarım zamanı
count'un değeri	Yeni değere bağlanır	Çalışma zamanı (yürütme sırasında)

5.4.1 Statik ve Dinamik Bağlama

Bağlama Türü	Tanım	Özellikler
Statik Bağlama (Static Binding)	Çalışma zamanı başlamadan önce gerçekleşir ve program yürütme boyunca değişmez.	• Derleme zamanında belirlenir • Değiştirilemez • Verimli
Dinamik Bağlama (Dynamic Binding)	Çalışma zamanında ilk kez gerçekleşir veya program yürütme sırasında değişebilir.	• Çalışma zamanında belirlenir • Değişebilir • Daha esnek ama maliyetli

5.4.2 Tür Bağlamaları (Type Bindings)

Bir değişken bir programda başvurulmadan önce bir veri türüne bağlanmış olmalıdır. Bu bağlamanın iki önemli yönü vardır: türün nasıl belirtildiği ve bağlamanın ne zaman gerçekleştiği.

5.4.2.1 Statik Tür Bağlama (Static Type Binding)

Bildirim Türü	Açıklama	Örnek
Açık Bildirim (Explicit Declaration)	Değişken isimlerini ve türlerini listeleyen bir program deyimidir.	int count; double sum;
Örtük Bildirim (Implicit Declaration)	İlk görünüşte varsayılan kurallara göre değişkeni türe bağlar; ayrı bir bildirim gerekmez.	Fortran: I-N ile başlayan isimler integer Perl: \$, @, %
Tür Çıkarımı (Type Inference)	Bağlam (context) ya da atanan değer kullanılarak tür çıkarılır.	C#: var sum = 0; → int var name = "Ali"; → string

Tür Çıkarımı Kullanan Diller: Visual Basic, C# (var), Swift, ML, Haskell, OCaml, F#

```
// C# – Tür Çıkarımı Örneği
var sum = 0; // int olarak çıkarılır
var total = 0.0; // float olarak çıkarılır
var name = "Fred"; // string olarak çıkarılır

// NOT: Bu değişkenler statik olarak tiplenmiştir –
// türleri bildirildikleri birimin ömrü boyunca sabittir.
```

⚠ Örtük Bildirimlerin Dezavantajı

Örtük bildirimler programcıya küçük bir kolaylık sağlasa da güvenilirliği olumsuz etkiler. Derleme süreci bazı yazım hatalarını ve programcı hatalarını yakalayamaz. Örneğin Fortran'da yanlış yazılan bir değişken ismi hata olarak değil, yeni bir değişken olarak yorumlanır.

5.4.2.2 Dinamik Tür Bağlama (Dynamic Type Binding)

Dinamik tür bağlamada, değişkenin türü bir bildirim deyimiyle belirtilmez ve ismin yazımından da belirlenemez. Bunun yerine değişken, bir atama deyiminde değer atandığında türe bağlanır. Atama gerçekleştiğinde, atanan değişken sağ taraftaki ifadenin türüne bağlanır.

```
// JavaScript – Dinamik Tür Bağlama Örneği
list = [10.2, 3.5]; // list şimdi 2 elemanlı sayısal dizi
list = 47; // list şimdi skaler değişken!

// Python örneği:
x = 42 # x → integer
x = "merhaba" # x → string (tür değişti!)
x = [1, 2, 3] # x → liste
```

Dinamik Tür Bağlama	Avantajlar	Dezavantajlar
Esneklik	Genel (generic) programlar yazılabilir; herhangi türde veri işlenebilir	-
Güvenilirlik	-	Derleyicinin hata tespiti azalır; yanlış tür ataması hata vermez
Performans	-	Her değişken için çalışma zamanı tanımlayıcısı gerekir; daha yavaş
Uygulama	-	Genellikle saf yorumlayıcı (pure interpreter) gerektirir; derleme zor

□ Dinamik Tür Bağlama Kullanan Modern Diller

Python, Ruby, JavaScript ve PHP dinamik tür bağlama kullanır. C# 2010'dan itibaren `dynamic` anahtar kelimesiyle isteğe bağlı dinamik tür bağlamayı destekler. Saf nesne yönelimli dillerde (örn. Ruby) tüm değişkenler referanstır ve türü yoktur; tüm veriler nesnedir.

⚠ Dinamik Tür Bağlamada Hata Tespiti Sorunu

JavaScript'te `i` ve `x` skalar, `y` dizi olsun. Programcı `i = x` yazacakken yanlışlıkla `i = y` yazdı. JavaScript'te bu bir hata değil, sadece `i`'nin türü dizi olarak değişir. Java'da ise derleyici bu hatayı derleme sırasında yakalar.

5.4.3 Depolama Bağlamaları ve Ömür (Storage Bindings and Lifetime)

Bir zorunlu programlama dilinin temel karakterini büyük ölçüde değişkenlerinin depolama bağlamalarının tasarımı belirler. Bir değişkenin bağlandığı bellek hücresi, mevcut bellek havuzundan alınmalıdır. Bu işleme tahsis (allocation), belleğin havuza geri iade edilmesine ise serbest bırakma (deallocation) denir.

▣ Değişken Ömrü (Variable Lifetime)

Bir değişkenin ömrü, değişkenin belirli bir bellek konumuna bağlı olduğu süre boyunca geçen zamandır. Ömür, değişken belirli bir hücreye bağlandığında başlar ve o hücreden bağı kesildiğinde sona erer.

Skaler (yapılandırılmamış) değişkenler ömürlerine göre dört kategoriye ayrılır:

KATEGORİ 1: Statik Değişkenler (Static Variables)

Statik değişken, program yürütmesi başlamadan önce bir bellek hücresine bağlanan ve program yürütmesi sona erene kadar aynı bellek hücresine bağlı kalan değişkendir.

```
// C/C++ – Statik Değişken Örneği
void subprogram() {
    static int call_count = 0; // Statik değişken
    call_count++;
    printf("Bu fonksiyon %d kez çağrıldı\n", call_count);
}
// call_count her çağrıda HATIRLANIR (history-sensitive)
```

Avantajlar	Dezavantajlar
✓ Verimli — doğrudan adresleme (direct addressing)	✗ Esneklik azalır — özyinelemeli (recursive) alt programları destekleyemez
✓ Tahsis/serbest bırakma için çalışma zamanı maliyeti yok	✗ Depolama birden fazla değişken arasında paylaşılamaz
✓ Tarihsel veri saklama (history-sensitive) mümkün	✗ Depolama esnekliği çok düşük

KATEGORİ 2: Yığın-Dinamik Değişkenler (Stack-Dynamic Variables)

Yığın-dinamik değişkenler, bildirim deyimleri ayrıntılandırıldığında (elaborated) depolama bağlamaları oluşturulan, ancak türleri statik olarak bağlanan değişkenlerdir. Ayrıntılandırma (elaboration), bildirimin eklendiği koda çalışma zamanında ulaşıldığında gerçekleşen depolama tahsis ve bağlama sürecini ifade eder.

```
// Java – Yığın-Dinamik Değişken Örneği
void hesapla() {
    int toplam = 0; // Metod çağrıldığında yığında tahsis edilir
    double oran = 0.5; // Metod bittiğinde serbest bırakılır
    // ...
}
// Özyinelemeli her çağrı kendi toplam ve oran kopyasına sahiptir!
```

Java, C++ ve C#'da değişken bildirimleri herhangi bir deyimin görünebileceği yerde yapılabilir. Bu dillerde yöntemler içindeki değişkenler varsayılan olarak yığın-dinamiktir.

Avantajlar	Dezavantajlar
✓ Özyinelemeli alt programlar mümkün (her aktif kopyanın kendi yerel değişkenleri var)	✗ Tahsis/serbest bırakma çalışma zamanı maliyeti (küçük ama var)
✓ Tüm alt programlar yerel değişkenler için aynı bellek alanını paylaşır	✗ Dolaylı adresleme (indirect addressing) gerekebilir → daha yavaş erişim
	✗ Alt programlar tarihsel veri saklayamaz (history-sensitive olamaz)

KATEGORİ 3: Açık Heap-Dinamik Değişkenler (Explicit Heap-Dynamic Variables)

Açık heap-dinamik değişkenler, programcının yazdığı açık çalışma zamanı komutlarıyla heap'ten tahsis edilen ve serbest bırakılan isimsiz (nameless/abstract) bellek hücreleridir. Bu değişkenlere yalnızca pointer veya referans değişkenler aracılığıyla erişilebilir. Heap, kullanımının öngörülemezliği nedeniyle son derece düzensiz organize edilmiş bir depolama hücreleri koleksiyonudur.

```
// C++ – Açık Heap-Dinamik Değişken Örneği
int *intnode; // Pointer değişkeni oluştur
intnode = new int; // Heap-dinamik değişkeni oluştur (new operatörü)
*intnode = 42; // Değer ata
// ...
delete intnode; // Değişkeni açıkça serbest bırak

// Java – Açık Heap-Dinamik (implicit GC)
String s = new String("Merhaba"); // new ile heap'te oluştur
// delete yok! Java otomatik çöp toplama (garbage collection) kullanır
```

Dil	Tahsis Yöntemi	Serbest Bırakma
C++	new operatörü	delete operatörü (açık, programcı sorumlu)
Java	new operatörü	Örtük çöp toplama (implicit garbage collection)
C#	new operatörü	Örtük (GC), ayrıca C++-tarzı pointer destekler
C	malloc() fonksiyonu	free() fonksiyonu (açık, programcı sorumlu)

⚠ Açık Heap-Dinamik Değişkenlerin Dezavantajları

- Pointer ve referans değişkenlerin doğru kullanımı zordur
- Değişkenlere başvuruların maliyeti yüksektir
- Gerekli depolama yönetimi gerçekleştirimi karmaşık ve maliyetlidir
- Kullanışlı oldukları yer: Bağlı listeler ve ağaçlar gibi büyüyüp küçülebilen dinamik yapılar

KATEGORİ 4: Örtük Heap-Dinamik Değişkenler (Implicit Heap-Dynamic Variables)

Örtük heap-dinamik değişkenler, yalnızca değer atandıklarında heap depolamasına bağlanır. Tüm özellikleri her atama yapıldığında yeniden bağlanır.

```
// JavaScript – Örtük Heap-Dinamik Örnek
highs = [74, 84, 86, 90, 71];
// highs → 5 elemanlı sayısal dizi (önceki türü ne olursa olsun)

highs = "Merhaba!";
// Şimdi highs → string (tüm özellikler yeniden bağlandı)
```

Örtük Heap-Dinamik	Avantaj	Dezavantaj
Esneklik	En yüksek esneklik derecesi — yüksek oranda genel (generic) kod yazılabilir	-
Güvenilirlik	-	Derleyicinin hata tespiti daha da azalır
Performans	-	Tüm dinamik özelliklerin çalışma zamanı maliyeti; dizi indis türleri ve aralıkları dahil

Dört Değişken Kategorisinin Karşılaştırması

Kategori	Tür Bağlama	Depolama Bağlama	Ömür	Örnek Diller
Statik	Statik	Program başlamadan önce	Program boyunca	C static, Fortran global
Yığın-Dinamik	Statik	Ayrıntılandırma zamanı	Alt program aktif olduğu süre	Java, C++, C# metodları
Açık Heap-Dinamik	Statik	Açık new/malloc ile	delete/free çağrısına kadar	C/C++ pointers, Java nesneleri
Örtük Heap-Dinamik	Dinamik	Atama sırasında	Sonraki atamalara kadar	JavaScript, Python, Ruby

Çıkma İhtimali Yüksek Sorular ve Cevapları

Olası Sınav Soruları

S1. Programlama dillerinde isim (name) tasarımındaki iki temel soruyu açıklayınız ve her biri için örnek veriniz.

Ç Cevap:

1. Büyük/Küçük Harf Duyarlılığı (Case Sensitivity): Dildeki isimler büyük/küçük harfe duyarlı mı? C, C++, Java ve C#'da büyük/küçük harf duyarlıdır: Rose, rose ve ROSE üç farklı isimdir. Bu durum okunabilirliği azaltır çünkü benzer görünen isimler farklı anlam taşır.
2. Ayrılmış Kelimeler mi, Anahtar Kelimeler mi?: Özel kelimeler programcı tarafından yeniden tanımlanabilir mi? Ayrılmış kelimeler (reserved words) yeniden tanımlanamaz (C, Java'da if, while). Anahtar kelimeler (keywords) ise bağlama göre özel anlam taşır ve yeniden tanımlanabilir (Fortran'da INTEGER gibi).

S2. Bir değişkeni karakterize eden altı özelliği (sextuple) sıralayınız ve her birini kısaca açıklayınız.

Ç Cevap:

1. İsim (Name): Değişkeni programda tanımlayan karakter dizisi.
2. Adres (Address): Değişkenin ilişkilendirildiği makine belleği adresi (l-değeri).
3. Değer (Value): Değişkenle ilişkili bellek hücresinin içeriği (r-değeri).
4. Tür (Type): Değişkenin depolayabileceği değer aralığını ve işlem kümesini belirler.
5. Ömür (Lifetime): Değişkenin belirli bir bellek konumuna bağlı olduğu süre.
6. Kapsam (Scope): Değişkenin görünür olduğu program deyimlerinin aralığı.

S3. Alias (takma ad) kavramını tanımlayınız. Alias'ın neden okunabilirliği ve güvenilirliği azalttığını açıklayınız ve C++'da alias oluşturulabilen üç yolu belirtiniz.

Ç Cevap:

Alias: Aynı bellek konumuna erişmek için birden fazla değişken ismi kullanılabildiğinde bu değişkenler birbirinin alias'ı olur.

Okunabilirliğe etkisi: Bir değişkenin değeri farklı bir isimle değiştirilebilir. Program okuyucusu her zaman hangi isimlerin aynı bellek hücrelerini paylaştığını hatırlamak zorundadır, bu da son derece güçtür.

Güvenilirliğe etkisi: Program doğrulamasını (verification) zorlaştırır.

C++'da Alias Oluşturma Yolları:

1. Union türleri: farklı türdeki alanlar aynı belleği paylaşır
2. Pointer değişkenler: iki pointer aynı konuma işaret edebilir
3. Referans değişkenler: `int &ref = x;` — ref ve x alias olur

S4. Bağlama (binding) ve bağlama zamanı (binding time) kavramlarını tanımlayınız. `count = count + 5`; C++ deyiminde en az üç farklı bağlamayı bağlama zamanlarıyla birlikte açıklayınız.

□ **Cevap:**

Bağlama: Bir özellik ile bir varlık arasındaki ilişkilendirmedir.

Bağlama Zamanı: Bu ilişkilendirmenin gerçekleştiği zandır.

`count = count + 5` deyimi için:

- `count`'un türü → derleme zamanında bağlanır
- `+` sembolünün anlamı (tamsayı toplama) → derleme zamanında (operand türleri bilinince)
- `5`'in iç gösterimi → derleyici tasarım zamanında
- `count`'un değeri → çalışma zamanında (atama yürütülünce)

S5. Statik ve dinamik tür bağlamayı karşılaştırınız. Her birinin avantajlarını ve dezavantajlarını belirtiniz. Hangi diller dinamik tür bağlama kullanır?

□ **Cevap:**

Statik Tür Bağlama: Tür derleme zamanında belirlenir ve değişmez. Avantajları: verimli, güvenilir, hataları erken tespit eder. Dezavantajı: esneklik az — genel (generic) programlar yazmak güçtür.

Dinamik Tür Bağlama: Tür atama sırasında çalışma zamanında belirlenir. Avantajı: yüksek esneklik, genel kod yazmak kolay. Dezavantajları: (1) güvenilirlik düşer — derleyici hata tespiti azalır, (2) maliyet artar — her değişken için çalışma zamanı tanımlayıcısı gerekir, (3) genellikle saf yorumlayıcı gerektirir. Dinamik tür bağlama kullanan diller: Python, Ruby, JavaScript, PHP. C# ise isteğe bağlı olarak `dynamic` anahtar kelimesiyle dinamik bağlamayı destekler.

S6. Dört deęişken ömür kategorisini (static, stack-dynamic, explicit heap-dynamic, implicit heap-dynamic) tanımlayınız ve her biri için avantaj/dezavantajlarını tartışınız.

□ **Cevap:**

1. Statik: Program başlamadan belleęe bağlanır, program bitene kadar bağli kalır. Avantaj: verimli doğrudan adresleme, tarihsel veri saklama. Dezavantaj: özyineleme desteklenemez, bellek paylaşımı yok.
2. Yığın-Dinamik: Alt program çağrıldığında yığında tahsis edilir, bitince serbest bırakılır. Avantaj: özyinelemeye izin verir, bellek paylaşımı. Dezavantaj: dolaylı adresleme maliyeti, tarihsel veri saklanamaz.
3. Açık Heap-Dinamik: Programcının açık talimatlarıyla heap'ten tahsis edilir/serbest bırakılır. Avantaj: dinamik yapılar (liste, ağaç) için ideal. Dezavantaj: pointer kullanımı zor, karmaşık bellek yönetimi.
4. Örtük Heap-Dinamik: Atama sırasında otomatik olarak heap'e bağlanır. Avantaj: maksimum esneklik. Dezavantaj: çalışma zamanı maliyeti yüksek, hata tespiti en düşük.

S7. Açık ve örtük tür bildirimlerini (explicit/implicit declarations) karşılaştırınız. Tür çıkarımı (type inference) nedir? Hangi diller kullanır?

□ **Cevap:**

Açık Bildirim: Deęişken isimlerini ve türleri listeleyen program deyimidir (`int x;`). Okunabilirliği ve güvenilirliği artırır.

Örtük Bildirim: İsimlerin sözdizimsel biçimine veya bağlama göre tür belirlenir. Yazımı kolaylaştırır ancak güvenilirliği azaltır — bazı hatalar derleyici tarafından yakalanamaz.

Tür Çıkarımı: Atanan deęerin türünden ya da bağlamdan deęişkenin türünü otomatik olarak belirleme yöntemidir. Statik bağlama sağlar ancak açık bildirim gerektirmez. Örnek: `var sum = 0;` → C#'da `int` olarak çıkarılır. Kullanan diller: C# (var), Visual Basic, Swift, ML, Haskell, OCaml, F#.

S8. C++ ve Java'da yığın-dinamik deęiřkenlerin iřleyiřini karřılařtırınız. Neden Java'da varsayılan deęiřkenler yığın-dinamiktir?

□ **Cevap:**

Her iki dilde de yöntem/fonksiyon içindeki deęiřkenler varsayılan olarak yığın-dinamiktir. Yöntem/fonksiyon çağrıldığında yığında yer ayrılır, tamamlandığında serbest bırakılır.

Farklar:

- C++ statik deęiřkene `static` belirteci ile izin verir; Java'da bu farklı çalışır (sınıf deęiřkeni anlamına gelir).
- Java'da tüm ilkel olmayan türler (nesneler) heap'te oluşturulur ve referans üzerinden erişilir.

Yığın-dinamik olmanın temel faydası özyinelemeyi desteklemesidir: her özyinelemeli çağrı kendi yerel deęiřkenlerinin ayrı bir kopyasına sahiptir.

❏ Kitap Sonu Soruları (Review Questions) ve Çözümleri

Bölüm 5.1–5.4 ile İlgili Sorular

Soru 1: İsimler (names) için tasarım sorunları nelerdir?

❏ Çözüm:

İki temel tasarım sorusu vardır:

1. İsimler büyük/küçük harf duyarlı mı? (Örn. C++'da evet; Fortran'da hayır.)
2. Dilin özel kelimeleri ayrılmış kelimeler mi (reserved words) yoksa anahtar kelimeler mi (keywords)? Ayrılmış kelimeler yeniden tanımlanamaz; anahtar kelimeler ise belirli bağlamlarda özel anlam taşır ama yeniden tanımlanabilir.

Soru 2: Büyük/küçük harf duyarlı isimlerin olası tehlikesi nedir?

❏ Çözüm:

Büyük/küçük harf duyarlılığının potansiyel tehlikesi okunabilirlik sorunudur. Benzer görünen isimler gerçekte tamamen farklı varlıkları temsil edebilir. Örneğin rose, Rose ve ROSE C++'da üç farklı değişkendir. Benzer görünen dil yapılarının benzer anlam taşıması gerektiği tasarım ilkesini ihlal eder. Ayrıca parseInt gibi Java'da önceden tanımlı isimlerin büyük/küçük harf kullanımını hatırlamak yazılabilirlik sorununa yol açar.

Soru 3: Alias nedir?

❏ Çözüm:

Birden fazla değişken ismi aynı bellek konumuna erişmek için kullanılabildiğinde, bu değişkenler birbirinin alias'ı (takma adı) olarak adlandırılır. Alias, okunabilirliği engeller ve program doğrulamasını güçleştirir. Örneğin total ve sum alias ise, birindeki değişiklik diğerini de etkiler.

Soru 4: C++ referans değişkenlerinin hangi kategorisi her zaman alias üretir?

❏ Çözüm:

C++'da referans değişkenleri (reference variables) her zaman alias oluşturur. Bir C++ pointer'ı isimli bir değişkeni işaret edecek şekilde ayarlandığında, pointer (dereference edildiğinde) ve değişkenin ismi alias olur. Benzer şekilde referans değişkenleri de her zaman alias üretir: `int &ref = x;` ifadesinde ref ve x alias'tır.

Soru 5: Bir deęişkenin l-deęeri nedir? r-deęeri nedir?

□ **Çözüm:**

l-deęeri (l-value): Deęişkenin adresidir. Deęişkenin ismi bir atama ifadesinin sol tarafında görüldüğünde gerekli olan deęerdir. "l" harfi "left" (sol) anlamına gelir.

r-deęeri (r-value): Deęişkenin deęeridir (bellek hücresinin içerięi). Deęişkenin ismi bir atama ifadesinin saę tarafında görüldüğünde gerekli olan deęerdir. "r" harfi "right" (saę) anlamına gelir. r-deęerine erişmek için önce l-deęerinin belirlenmesi gerekir.

Soru 6: Baęlama (binding) ve baęlama zamanını (binding time) tanımlayınız.

□ **Çözüm:**

Baęlama (Binding): Bir özellik ile bir varlık arasındaki ilişkilendirmedir. Örneęin bir deęişkenin türüyle ya da deęeriyle, veya bir işlemle sembolü arasındaki ilişkilendirmedir.

Baęlama Zamanı (Binding Time): Baęlamanın gerçekleştięi zamandır. Baęlama zamanları şunlardır: dil tasarım zamanı, dil gerçekleştirme zamanı, derleme zamanı, yükleme zamanı, baęlama (link) zamanı ve çalışma zamanı. Baęlama ve baęlama zamanları, programlama dillerinin anlambiliminin (semantics) önde gelen kavramlarıdır.

Soru 7: Dil tasarımı ve gerçekleştirmesinden sonra bir programda baęlamaların gerçekleşebileceęi dört zaman nedir?

□ **Çözüm:**

1. Derleme zamanı (Compile time): Deęişkenin veri türüne baęlanması
2. Yükleme zamanı (Load time): Global deęişkenlerin depolama hücrelerine baęlanması
3. Baęlama zamanı / Link time: Kütüphane alt programına yapılan çağrının baęlanması
4. Çalışma zamanı (Run time): Java metodundaki yerel deęişkenin depolamaya baęlanması, veya dinamik tür baęlamada tür baęlanması

Soru 8: Statik baęlamayı (static binding) ve dinamik baęlamayı (dynamic binding) tanımlayınız.

□ **Çözüm:**

Statik Baęlama: Çalışma zamanı başlamadan önce ilk kez gerçekleşen ve program yürütme boyunca deęişmeyen baęlamadır. Derleme zamanında veya daha erken gerçekleşen tüm baęlamalar statiktir.

Dinamik Baęlama: Çalışma zamanında ilk kez gerçekleşen veya program yürütme sırasında deęişebilen baęlamadır. Daha fazla esneklik saęlar ancak okunabilirlik, verimlilik ve güvenilirlik pahasına.

Soru 9: Örtük bildirimlerin (implicit declarations) avantajları ve dezavantajları nelerdir?

□ **Çözüm:**

Avantajları:

- Programcıya küçük bir kolaylık sağlar — tür adı yazılması gerekmez
- Perl gibi dillerde farklı tür ad alanları oluşturarak (\$, @, %) okuyucuya tür bilgisi verir

Dezavantajları:

- Güvenilirliği azaltır — derleme süreci bazı yazım ve programcı hatalarını yakalayamaz
- Fortran'da yanlış yazılan değişken ismi hata değil, yeni bir değişken olarak yorumlanır
- Program bakımını güçleştirir

Soru 10: Dinamik tür bağlamanın (dynamic type binding) avantajları ve dezavantajları nelerdir?

□ **Çözüm:**

Avantajları:

- Daha fazla programlama esnekliği sağlar
- Genel (generic) programlar yazılabilir — hangi türde veri girilirse girsin işlenebilir

Dezavantajları:

- Güvenilirlik düşer: Derleyicinin hata tespiti azalır. Yanlış türdeki atamalar hata değil, tür değişimi olarak yorumlanır
 - Maliyet yüksektir: Tür denetimi çalışma zamanında yapılmalı, her değişken için çalışma zamanı tanımlayıcısı gerekli, farklı türler farklı bellek boyutu gerektirir
 - Uygulama zorluğu: Genellikle saf yorumlayıcı gerektirir (derleyici yazmak çok zordur), bu da performansı ciddi ölçüde düşürür
-

Soru 11: Statik, yığın-dinamik, açık heap-dinamik ve örtük heap-dinamik değişkenleri tanımlayınız. Avantajları ve dezavantajları nelerdir?

□ **Çözüm:**

Statik: Program başlamadan belleğe bağlanır; program bitene kadar sabit kalır. Avantaj: hızlı doğrudan adresleme, tarihsel veri saklama, tahsis maliyeti yok. Dezavantaj: özyineleme desteği yok, bellek paylaşımı yok, esneklik düşük.

Yığın-Dinamik: Alt program çağrısında yığında tahsis edilir, bitişte serbest bırakılır. Avantaj: özyinelemeye izin verir, alt programlar bellek paylaşır. Dezavantaj: dolaylı adresleme maliyeti, tarihsel veri saklanamaz.

Açık Heap-Dinamik: Programcının açık talimatlarıyla (new/malloc) heap'ten tahsis edilir. Avantaj: dinamik büyüyüp küçülebilen yapılar için idealdir. Dezavantaj: pointer/referans kullanımı zor, karmaşık bellek yönetimi.

Örtük Heap-Dinamik: Değer atandığında otomatik bağlanır, tüm özellikler yeniden bağlanır. Avantaj: en yüksek esneklik. Dezavantaj: yüksek çalışma zamanı maliyeti, derleyici hata tespiti en düşük.

▢ Bölüm Özeti — Temel Kavramlar

Hızlı Referans

Kavram	Tanım / Açıklama
İsim (Name)	Programda varlıkları tanımlamak için kullanılan karakter dizisi; tanımlayıcı (identifier) ile eş anlamlı
Ayrılmış Kelime	Yeniden tanımlanamayan özel dil kelimesi (if, while, for)
Anahtar Kelime	Bağlama göre özel anlam taşıyan, yeniden tanımlanabilir kelime (Fortran)
Değişken Altılısı	(isim, adres, değer, tür, ömür, kapsam) — 6 temel özellik
Alias	Aynı bellek konumuna erişen birden fazla değişken ismi
l-değeri	Değişkenin adresi; atama solunda kullanılır
r-değeri	Değişkenin içeriği/değeri; atama sağında kullanılır
Bağlama	Bir özellik ile bir varlık arasındaki ilişkilendirme
Statik Bağlama	Çalışma öncesi gerçekleşir, değişmez
Dinamik Bağlama	Çalışma zamanında gerçekleşir veya değişebilir
Açık Bildirim	Tür ve ismi açıkça belirten bildirim deyimi
Örtük Bildirim	Varsayılan kurallara göre otomatik tür belirleme
Tür Çıkarımı	Bağlam veya atanan değerden türü otomatik belirleme
Ömür (Lifetime)	Değişkenin belirli bellek konumuna bağlı olduğu süre
Statik Değişken	Program başından sonuna kadar aynı belleğe bağlı
Yığın-Dinamik	Alt program çağrısında yığında tahsis edilir
Açık Heap-Dinamik	new/malloc ile heap'ten tahsis; delete/free ile serbest
Örtük Heap-Dinamik	Atama sırasında otomatik heap'e bağlanır